AWS VPC Peering Project Report

Private communication between two Amazon Virtual Private Clouds

Project Name	AWS VPC Peering Implementation
Internship Organization	EWB Edutech Private Limited (EWB Courses)
Internship Track	Amazon Web Services (AWS) Cloud Computing Internship
AWS Region	Asia Pacific (Mumbai) — ap-south-1
Report Type	Sanitized public portfolio copy

Public-sharing notice: This report intentionally excludes personal registration numbers, application reference numbers, AWS account identifiers, public IP addresses, unique AWS resource IDs, and credentials. It is prepared for a public GitHub portfolio.

Prepared as a project record of the AWS networking implementation and validation activities.

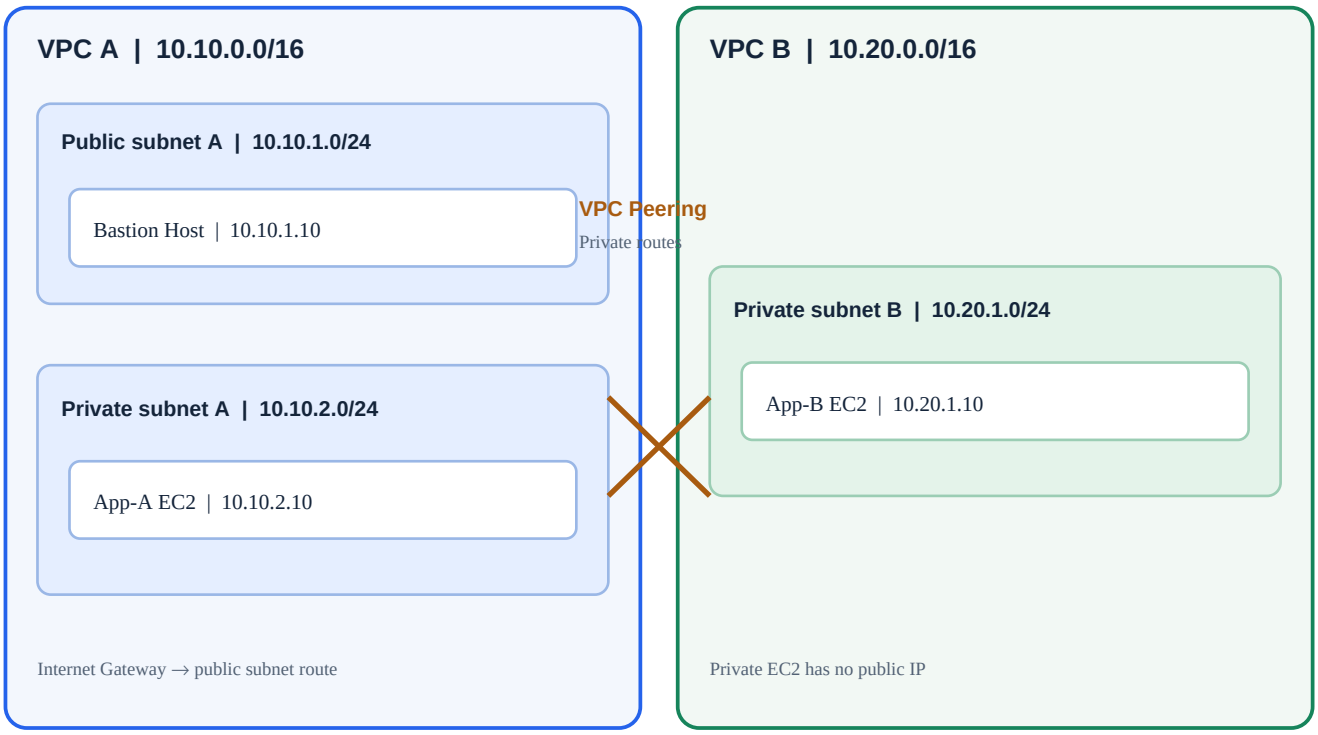
1. Introduction

What is Amazon VPC? Amazon Virtual Private Cloud (Amazon VPC) lets a user create a logically isolated virtual network in AWS. The user defines network ranges, subnets, route tables, and security controls for resources deployed in that network.

What is VPC Peering? VPC Peering is a networking connection between two VPCs that allows resources in those VPCs to communicate using private IP addresses, subject to route-table entries and network security rules. The VPCs in this project use non-overlapping CIDR ranges so that their address spaces can be routed separately.

Project objective: Create two VPCs in the Mumbai AWS Region, connect them through a VPC Peering connection, deploy private EC2 application instances in each VPC, and validate communication between them. A Bastion Host in the public subnet provides an SSH entry point to the private instances.

2. Architecture



Architecture explanation

VPC A contains a public subnet for the Bastion Host and a private subnet for App-A. VPC B contains a private subnet for App-B. The VPC Peering connection links the VPCs. Route tables direct traffic for the remote VPC CIDR to the peering connection. The Internet Gateway is attached to VPC A to support access to the Bastion Host; the application instances remain private.

The diagram shows the logical design and sanitized network addressing. Unique AWS resource identifiers and the Bastion Host public endpoint are deliberately not shown.

3. AWS Configuration

3.1 VPCs and subnets

Resource	Purpose	CIDR block
VPC A — ewb-vpc-a	Network hosting Bastion and App-A	10.10.0.0/16
VPC B — ewb-vpc-b	Network hosting App-B	10.20.0.0/16
Public subnet A	Bastion Host subnet	10.10.1.0/24
Private subnet A	App-A subnet	10.10.2.0/24
Private subnet B	App-B subnet	10.20.1.0/24

3.2 EC2 instances

Instance	Role	Private IP	Placement
Bastion Host	SSH entry point	10.10.1.10	Public subnet A
App-A	Private application host	10.10.2.10	Private subnet A
App-B	Private application host	10.20.1.10	Private subnet B

3.3 Route tables and VPC Peering

A VPC Peering connection was created between VPC A and VPC B and brought to the active state. The route tables were updated so that traffic destined for the other VPC's CIDR is directed to the peering connection. In summary:

• VPC A public route table: local VPC A route, default route to the Internet Gateway, and route for 10.20.0.0/16 through the peering connection.
• VPC A private route table: local VPC A route and route for 10.20.0.0/16 through the peering connection.
• VPC B private route table: local VPC B route and route for 10.10.0.0/16 through the peering connection.

3.4 Security Groups

Security Groups were configured to allow only the required management and test traffic. The Bastion Host permits SSH from the administrator's current public IP address (not reproduced in this public report). App-A and App-B permit SSH from the Bastion Host's private address and ICMP Echo Request from the opposite application instance's private address. Default outbound access was retained for the lab test.

3.5 Other AWS components

The environment used Amazon EC2 instances running Amazon Linux 2023, an Internet Gateway for the public subnet in VPC A, route tables for traffic direction, and Security Groups for instance-level traffic filtering.

4. Implementation Process

The environment was built in the following order:

01. Create VPCs

Create VPC A (10.10.0.0/16) and VPC B (10.20.0.0/16), using non-overlapping CIDR blocks.

02. Create subnets

Create the public and private subnets in VPC A and the private subnet in VPC B.

03. Configure internet access for Bastion

Attach an Internet Gateway to VPC A and add the default internet route to the public route table.

04. Create route tables

Associate the route tables with the appropriate subnets and configure local and internet routes.

05. Launch EC2 instances

Launch the Bastion Host in the public subnet and App-A/App-B in their private subnets, without public IP addresses on the application instances.

06. Create VPC Peering

Create the peering request between the two VPCs and accept it so that the connection becomes active.

07. Add peering routes

Add each remote VPC CIDR to the relevant route tables with the VPC Peering connection as the target.

08. Configure Security Groups

Allow the required SSH and ICMP traffic from the intended source addresses.

09. Connect through Bastion

Connect to the Bastion Host and then SSH to a private EC2 instance using its private IP address.

10. Validate communication

Run ping tests between App-A and App-B, then perform a controlled Security Group failure and recovery test.

5. Connectivity Testing and Results

5.1 SSH access test

SSH access to the private application instances was performed through the Bastion Host. The workflow was: administrator workstation → Bastion Host in VPC A → private EC2 instance using its private IP address. SSH agent forwarding was used for the onward connection. The application instances did not need public IP addresses.

5.2 ICMP ping test

Ping tests were performed between App-A (10.10.2.10) and App-B (10.20.1.10) to validate private communication across the peering connection. The successful test result recorded during the lab was 4 replies received out of 4 packets (0% packet loss) for the verified direction after the security rules were correctly configured.

Test	Expected / observed result	Status
SSH via Bastion	SSH access to private EC2 through Bastion Host was successful.	Passed
ICMP ping after configuration	Successful replies; 4/4 packets, 0% loss in the verified test direction.	Passed

Evidence note

Console and terminal screenshots are intended to be provided separately in the repository's **screenshots/** folder. This report does not claim to contain embedded screenshots. Add sanitized screenshots of the SSH session and successful ping output to make the test evidence independently reviewable.

6. Failure Testing

A controlled negative test was performed to demonstrate the effect of Security Group inbound rules. The inbound ICMP Echo Request rule on App-A's Security Group, which allowed traffic from App-B (10.20.1.10), was temporarily removed. A ping attempt from App-B to App-A then failed, with packet loss observed during the test.

Why this happened: The peering connection and routes provide a network path, but the destination Security Group still controls whether inbound ICMP traffic is accepted. Removing the relevant inbound rule caused the destination to reject the ping traffic.

7. Recovery Testing

After the negative test, the inbound ICMP Echo Request rule on App-A's Security Group was restored with App-B's private IP address as the source. The ping test was repeated from App-B to App-A and successful replies were received (4/4 packets, 0% packet loss in the recorded recovery test).

Recovery conclusion: Restoring the Security Group rule restored the expected ICMP communication. This confirms that the observed failure was caused by the removed inbound rule and that the required peering route remained available.

Evidence required in the GitHub screenshots folder

Suggested filename	What the screenshot should show
10-ping-failure.png	Terminal output showing ping failure while the App-A inbound ICMP rule is removed.
11-ping-recovery.png	Terminal output showing successful ping after the App-A inbound ICMP rule is restored.
06-security-groups.png	Security Group inbound rules after restoration, with sensitive account/resource details redacted.

Important: The report records the test outcomes from the project work. The actual terminal screenshots must be uploaded separately; do not create or edit screenshots to make a test appear successful or failed.

8. Learning Outcomes

Through this implementation, the following practical concepts were learned:

VPC and CIDR planning: Designed two isolated VPC networks with non-overlapping CIDR blocks and understood how subnet ranges fit within a VPC range.

Subnet design: Separated public access infrastructure from private application instances by placing the Bastion Host in a public subnet and the application servers in private subnets.

VPC Peering: Created and activated a peering connection and understood that peering alone is not sufficient without correct routes and security rules.

Route-table configuration: Added routes for the peer VPC CIDR and learned how route-table targets determine the path for inter-VPC traffic.

Security Group management: Configured SSH and ICMP inbound rules using specific source addresses and observed how a missing inbound rule blocks traffic.

Bastion Host access: Connected to private EC2 instances through a Bastion Host rather than exposing the application instances directly to the internet.

Connectivity troubleshooting: Used SSH and ping tests to validate connectivity, isolate a Security Group issue, and confirm recovery after restoring the rule.

Cloud networking practice: Gained hands-on experience combining VPCs, subnets, route tables, peering, EC2, an Internet Gateway, and Security Groups in one AWS environment.

9. Conclusion

The project successfully demonstrated private communication between two AWS VPCs using VPC Peering. The application instances communicated through their private IP addresses after the required routes and Security Group rules were configured. The Bastion Host provided a controlled SSH path to private instances. The failure and recovery exercise showed the importance of Security Group inbound rules when troubleshooting network connectivity.

10. Public Repository Security Checklist

Before publishing screenshots or supporting files, verify that they do not reveal personal registration identifiers, application reference numbers, AWS account IDs, public IP addresses, unique resource IDs, private keys, passwords, access keys, session tokens, or other credentials. Never upload a **.pem** private key file. Keep the detailed, unredacted submission report private and use this sanitized version for the public repository.

Project details: AWS VPC Peering | EWB Edutech Private Limited (EWB Courses) | AWS Cloud Computing Internship | ap-south-1